

Tensorflow II

Convolutional Neural Networks (CNNs)

Fatemeh Dehrouyeh



- 1 Convolutional Neural Networks
 - From Image Processing to CNNs
 - Convolution Layers
 - Stride and Padding
 - Feature Maps
 - Activation Functions
 - Pooling

Current Section

1 Convolutional Neural Networks

- From Image Processing to CNNs
- Convolution Layers
- Stride and Padding
- Feature Maps
- Activation Functions
- Pooling

Current Subsection

- 1 Convolutional Neural Networks
 - From Image Processing to CNNs
 - Convolution Layers
 - Stride and Padding
 - Feature Maps
 - Activation Functions
 - Pooling

From Classical Filters to Learned Filters

In classical image processing, we manually choose filters:

blur, sharpen, edge detection

In CNNs, the model learns useful filters automatically from data.

Core Idea

A convolutional neural network learns visual patterns directly from images.

Pixels $\rightarrow$ Features $\rightarrow$ Objects $\rightarrow$ Prediction

What Is a CNN?

A **Convolutional Neural Network** is a neural network designed for image data.

CNNs use layers that scan across an image and detect local patterns.

- early layers detect simple patterns such as edges,
- middle layers detect textures and parts,
- deeper layers detect object-level patterns.

Why CNNs Work Well for Images

Images have local structure: nearby pixels are strongly related.

The Main Building Blocks of a CNN

A basic CNN is usually built from:

- **convolution layers:** learn visual filters,
- **activation functions:** add nonlinearity,
- **pooling layers:** reduce spatial size,
- **flattening or global pooling:** convert features,
- **dense layers:** produce predictions.

Image $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ Dense $\rightarrow$ Output

Code

```

▶ import tensorflow as tf
import matplotlib.pyplot as plt

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16, kernel_size=3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, kernel_size=3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.summary()

```

... Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 16)	448
max_pooling2d (MaxPooling2D)	(None, 111, 111, 16)	0
conv2d_1 (Conv2D)	(None, 109, 109, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 32)	0
flatten (Flatten)	(None, 93312)	0
dense (Dense)	(None, 10)	933,130

Total params: 938,218 (3.58 MB)
 Trainable params: 938,218 (3.58 MB)
 Non-trainable params: 0 (0.00 B)

Current Subsection

1 Convolutional Neural Networks

- From Image Processing to CNNs
- **Convolution Layers**
- Stride and Padding
- Feature Maps
- Activation Functions
- Pooling

What Does a Convolution Layer Do?

A convolution layer applies multiple learnable filters to an image.

Each filter scans across the image and produces a **feature map**.

Image $\rightarrow$ Learned Filter $\rightarrow$ Feature Map

Key Idea

A feature map shows where a certain visual pattern appears in the image.

Convolution Filters and RGB Channels

For an RGB image, the input has three channels:

$$H_{\text{in}} \times W_{\text{in}} \times 3$$

A convolution filter spans all input channels.

So a 3×3 filter is actually

$$3 \times 3 \times 3$$

because it looks at red, green, and blue together.

At each spatial location, the filter combines all channel values into one number:

$$z = \sum_{c=1}^3 \sum_i \sum_j w_{ijc} x_{ijc} + b$$

As the filter slides over the image, these numbers form one feature map.

one filter $\rightarrow$ one feature map

Therefore, with

$$\text{Conv2D}(16, \text{kernel_size}=3)$$

the layer learns 16 filters and produces 16 feature maps:

Filters in a CNN

```
tf.keras.layers.Conv2D(filters=16, kernel_size=3, activation="relu")
```

In classical image processing, filters are manually designed.

In CNNs, filters are initialized randomly and then learned during training.

For example, a convolution layer with

$$\text{filters} = 16$$

learns 16 different filters and produces 16 feature maps.

Interpretation

Each filter can learn to respond to a different visual pattern.

Kernel Size

The **kernel size** controls the spatial size of the filter.

Common examples:

$$3 \times 3, \quad 5 \times 5, \quad 7 \times 7$$

A 3×3 kernel looks at a small local neighborhood.

A larger kernel sees a larger region at once.

Common Practice

Many CNNs use 3×3 kernels because they are efficient and can be stacked to learn complex features.

Current Subsection

1 Convolutional Neural Networks

- From Image Processing to CNNs
- Convolution Layers
- **Stride and Padding**
- Feature Maps
- Activation Functions
- Pooling

What Is Stride?

Stride controls how far the convolution filter moves at each step.

For example:

$$\text{stride} = 1$$

moves the filter one pixel at a time.

$$\text{stride} = 2$$

moves the filter two pixels at a time.

Effect of Larger Stride

A larger stride reduces the spatial size of the output feature map.

What Is Padding?

Convolution near the image boundary has fewer neighboring pixels available.

Padding controls what happens at the borders.

- `valid`: no padding
- `same`: padding is added so the output size is preserved when stride is 1

Why Padding Matters

Without padding, feature maps become smaller after each convolution.

Output of a Convolution Layer

For a convolution layer, the spatial output size is determined by the input size, filter size, padding, and stride.
For an input of size

$$H_{\text{in}} \times W_{\text{in}} \times C_{\text{in}},$$

the output height and width are

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P_h - K_h}{S_h} \right\rfloor + 1,$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P_w - K_w}{S_w} \right\rfloor + 1.$$

Here,

$$K_h, K_w = \text{filter size}, \quad S_h, S_w = \text{stride}, \quad P_h, P_w = \text{padding}.$$

The output depth is determined by the number of filters:

$$C_{\text{out}} = N_{\text{filters}}.$$

Therefore, the complete output size is

$$H_{\text{out}} \times W_{\text{out}} \times N_{\text{filters}}.$$

For

$$\text{Conv2D}(16, \text{kernel_size}=3, \text{activation}=\text{"relu"}),$$

Keras uses stride $S = 1$ and valid padding $P = 0$ by default. For an input of $224 \times 224 \times 3$,

$$H_{\text{out}} = W_{\text{out}} = \frac{224 + 2(0) - 3}{1} + 1 = 222.$$

Thus, the output size is

$$222 \times 222 \times 16.$$

After the convolution and bias are computed, ReLU is applied element-wise:

$$z = \sum w_j x_i + b, \quad \text{ReLU}(z) = \max(0, z).$$

Therefore, the activation changes the values of the feature map, but not its dimensions.

Current Subsection

1 Convolutional Neural Networks

- From Image Processing to CNNs
- Convolution Layers
- Stride and Padding
- **Feature Maps**
- Activation Functions
- Pooling

What Is a Feature Map?

A convolution filter produces an output called a **feature map**.

If a convolution layer contains

32

filters, it produces

32

feature maps.

Each feature map represents the response of one learned filter across the image.

Interpretation

Different feature maps can respond to different patterns such as edges, textures, colors, or shapes.

Visualizing Feature Maps

Feature maps help us inspect what a convolution layer responds to.

For one input image:

$$(H_{\text{in}}, W_{\text{in}}, 3)$$

a convolution layer might produce:

$$(H_{\text{out}}, W_{\text{out}}, 16)$$

The last dimension corresponds to different learned filters.

Loading Lena Image

```
import tensorflow as tf

# Load Lena image from GitHub
url = "https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg"
path = tf.keras.utils.get_file("lena.jpg", url)

# Read and decode image
img = tf.io.read_file(path)
img = tf.image.decode_jpeg(img, channels=3)

# Convert to float [0, 1]
img_float = tf.image.convert_image_dtype(img, tf.float32)

print(img_float.shape)
print(img_float.dtype)
```

... Downloading data from <https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg>
91814/91814 ————— 0s 0us/step
(512, 512, 3)
<dtype: 'float32'>

Lena Image

```
import matplotlib.pyplot as plt
```

```
plt.imshow(img_float)  
plt.axis("off")  
plt.show()
```

...



Code Example

```
▶ img = tf.image.resize(img_float, (224, 224))
img_batch = tf.expand_dims(img, axis=0)

conv_layer = tf.keras.layers.Conv2D(
    filters=8,
    kernel_size=3,
    padding="same",
    activation="relu"
)

feature_maps = conv_layer(img_batch)

print(feature_maps.shape)

fig, axes = plt.subplots(2, 4, figsize=(12, 6))

for i, ax in enumerate(axes.ravel()):
    ax.imshow(feature_maps[0, :, :, i], cmap="gray")
    ax.set_title(f"Feature Map {i+1}")
    ax.axis("off")

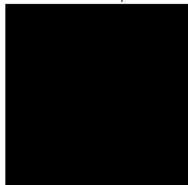
plt.tight_layout()
plt.show()
```

Result

(1, 224, 224, 8)

...

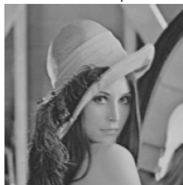
Feature Map 1



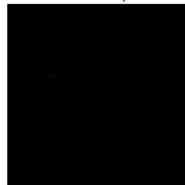
Feature Map 2



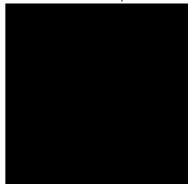
Feature Map 3



Feature Map 4



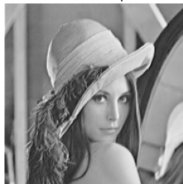
Feature Map 5



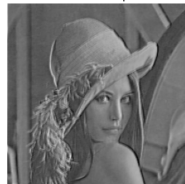
Feature Map 6



Feature Map 7



Feature Map 8



Current Subsection

1 Convolutional Neural Networks

- From Image Processing to CNNs
- Convolution Layers
- Stride and Padding
- Feature Maps
- **Activation Functions**
- Pooling

Why Do We Need an Activation Function?

Convolution itself is a linear operation.

If we stack only linear operations, the network remains limited.

An **activation function** introduces nonlinearity.

The most common activation in CNNs is:

ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Key Idea

ReLU keeps positive values and replaces negative values with zero.

Convolution + ReLU

A common CNN operation is:

Input $\rightarrow$ Convolution $\rightarrow$ ReLU $\rightarrow$ Feature Map

In TensorFlow, this can be written directly as:

```
Conv2D(..., activation="relu")
```

Interpretation

The convolution detects a pattern, while ReLU keeps strong positive responses.

Current Subsection

1 Convolutional Neural Networks

- From Image Processing to CNNs
- Convolution Layers
- Stride and Padding
- Feature Maps
- Activation Functions
- Pooling

What Is Pooling?

Pooling reduces the spatial size of feature maps.

It helps:

- reduce computation,
- reduce memory usage,
- summarize local features,
- make representations less sensitive to small shifts.

Most Common Type

Max pooling keeps the largest value from each local region.

Pooling Defaults

```
tf.keras.layers.MaxPooling2D(  
    pool_size=(2,2),  
    strides=None,  
    padding="valid"  
)
```

In Keras, `MaxPooling2D()` uses a 2×2 pooling window by default; when `strides=None`, the stride automatically matches the pooling size, so the effective stride is also 2×2 .

Max Pooling Example

Consider a 2×2 region:

$$\begin{bmatrix} 1 & 5 \\ 2 & 3 \end{bmatrix}$$

Max pooling keeps:

$$\boxed{5}$$

Using 2×2 pooling typically reduces:

$$222 \times 222 \rightarrow 111 \times 111$$

Key Idea

Pooling keeps strong local responses while reducing spatial resolution.

Convolution and Pooling Together

A typical CNN repeatedly combines convolution and pooling:

$\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool}$

As we move deeper:

- spatial dimensions usually become smaller,
- the number of feature maps often increases,
- learned features become more abstract.

$$222 \times 222 \times 3 \rightarrow 222 \times 222 \times 32 \rightarrow 111 \times 111 \times 32$$

How the Number of Parameters Is Computed

Layer	Output Shape	Parameters
Conv2D	$222 \times 222 \times 16$	448
MaxPooling2D	$111 \times 111 \times 16$	0
Conv2D	$109 \times 109 \times 32$	4640
MaxPooling2D	$54 \times 54 \times 32$	0
Flatten	93312	0
Dense	10	933130

$$\text{Conv2D}_1 : (3 \times 3 \times 3 + 1) \times 16 = 448$$

$$\underbrace{3 \times 3}_{\text{kernel size}} \times \underbrace{3}_{\text{RGB channels}} + \underbrace{1}_{\text{bias}} \quad \text{for each of } \underbrace{16}_{\text{filters}}$$

$$\text{Conv2D}_2 : (3 \times 3 \times 16 + 1) \times 32 = 4640$$

$$\underbrace{3 \times 3}_{\text{kernel size}} \times \underbrace{16}_{\text{input feature maps}} + \underbrace{1}_{\text{bias}} \quad \text{for each of } \underbrace{32}_{\text{filters}}$$

$$\text{Flatten} : 54 \times 54 \times 32 = 93312$$

$$\underbrace{54 \times 54}_{\text{spatial size}} \times \underbrace{32}_{\text{feature maps}} = 93312$$

$$\text{Dense} : (93312 + 1) \times 10 = 933130$$

$$\underbrace{93312}_{\text{input values}} + \underbrace{1}_{\text{bias}} \quad \text{for each of } \underbrace{10}_{\text{output neurons}}$$

$$\text{Total} = 448 + 4640 + 933130 = 938218$$

Congratulations!

Now you know basics of CNN!

